

PROJECT REPORT
ON GROCERY BILLING SYSTEM
STORE NAME : FRESH BASKET

Submitted by- Abhisarika Tripathi
Registration No.- 26BCE11411
Course - BTech Core

1. Introduction & Objectives :

The Fresh Basket Grocery Store Billing System is a desktop application developed using Python's robust GUI library, Tkinter. It provides a digital point-of-sale (POS) terminal tailored for small to medium-sized grocery stores. The application manages inventory selections, handles dynamic cart updates, validates customer credentials, and automates tax and billing receipt calculations.

2. Problem Statement:

Manual checkout systems result in prolonged customer wait times and clerical errors during calculations (such as miscalculating item totals, applicable taxes, or grand totals). Furthermore, without a clear, real-time breakdown of an order, store clerks struggle to update active customer baskets—like adding or removing specific quantities on the fly—leading to friction at the point of sale.

3. Functional Requirements

Functional requirements describe exactly what the system **must do** from a user's perspective.

A. User Management & Input Validation

- **Customer Info Entry:** The system must provide input fields to capture the customer's Name and Phone Number.
- **Input Validation:** The system must validate that the Name field is not empty and the Phone Number contains exactly 10 numerical digits before allowing checkout.

B. Inventory & Cart Management

- **Product Catalog:** The system must display a pre-loaded list of items along with their respective prices in Indian Rupees (₹).
- **Add to Cart:** The system must allow users to select an item, specify a quantity (from 1 to 20 using a spinbox), and append it to the cart listbox.
- **Remove from Cart:** The system must allow users to select any individual entry from the active cart list and delete it entirely.
- **Dynamic Subtotal Tracking:** The system must automatically recalculate and display the live subtotal every time an item is added or removed.

C. Billing & Invoice Generation

- **Empty Cart Prevention:** The system must reject a checkout request if no items have been added to the cart.
- **Taxation & Grand Total:** The system must automatically calculate a 5% tax on the subtotal and add it to the final payable amount.
- **Invoice Output:** The system must display a clean, structured pop-up message box serving as the digital receipt, containing customer details, an itemized breakdown, tax details, and the grand total.
- **State Reset:** The system must automatically clear the shopping cart and reset the input fields upon successful completion of a transaction.

4. Non Functional Requirements

Performance Requirements

- Instant UI Responsiveness: The user interface must update instantly (within under 100 milliseconds) when a user adds or removes an item from the cart, preventing any visual lag for the billing clerk.
- Rapid Invoice Computation: The checkout system must calculate items, apply taxes, and display the final bill pop-up window within 0.5 seconds of clicking the "Checkout" button.

Security Requirements

- Input Sanitization: The application must sanitize all data entered into the Customer Name and Phone Number fields to prevent malicious characters or code execution (such as script injection) from crashing the program.
- State Isolation: The active shopping cart data and customer personal information must be completely cleared from the system's temporary memory (RAM) immediately after a transaction is finalized or canceled.

Usability Requirements

- High Visual Readability: The interface must use clear, highly legible typography—such as 12pt to 16pt Times New Roman fonts—with distinct contrasting text colors (dark green and red buttons) to prevent eye strain during long store shifts.
- One-Screen Workspace Layout: All vital operational tools (customer details, item list, cart status, and action buttons) must fit onto a single 550x500 pixel window without requiring the clerk to scroll horizontally.

Reliability Requirements

- Zero Calculation Failures: The billing logic must achieve a 100% calculation accuracy rate, ensuring that cumulative sums, 5% tax applications, and grand totals contain no floating-point rounding errors.

- Graceful Empty State Recovery: If a clerk accidentally clicks "Add to Cart", "Remove Item", or "Checkout" without selecting data, the application must completely prevent crashes by intercepting the action with a friendly alert box.

5. PROJECT OBJECTIVES:

- To create an intuitive, user-friendly Graphical User Interface (GUI) for swift checkout operations.
- To prevent human billing errors by automating item cost evaluation, a 5% tax application, and grand total calculations.
- To implement input verification safeguards (such as confirming 10-digit mobile configurations and preventing blank entry fields).
- To allow quick interactive cart adjustments (adding inventory or removing selected items).

6. System Requirements

Hardware Requirements

- **Processor:** Intel Core i3 / AMD Ryzen 3 or higher
- **RAM:** 4 GB minimum (8 GB recommended)
- **Storage:** 50 MB available disk space

Software Requirements

- **Operating System:** Windows 10/11, macOS, or Linux
- **Programming Language:** Python 3.x
- **GUI Framework:** Tkinter (Included with standard Python installations)

7. Implementation Architectural design Documentation Requirements :

- Initializing Data and Variables :

This part of the code defines the available grocery products using a Python dictionary and creates an empty cart to track user selections.

```
C: > Users > hp > Downloads > grocery.py > checkout
1  import tkinter as tk
2  from tkinter import messagebox
3
4  items = {
5      1: ["Milk", 2.50],
6      2: ["Bread", 1.80],
7      3: ["Eggs", 3.20],
8      4: ["Chips", 10.50],
9      5: ["Chocolate", 5.50],
10     6: ["Apple", 4.00],
11     7: ["Jam", 10.00],
12     8: ["Curd", 15.65],
13 }
14 cart = {}
15
```

- Add to Cart Function

This function reads the item selected by the user, grabs the chosen quantity from the counter box, and adds it to the digital shopping cart.

```

15
16 def add_to_cart():
17     selected = item_list.curselection()
18     if not selected:
19         messagebox.showwarning("No item", "Please select an item")
20         return
21
22     item_id = list(items.keys())[selected[0]]
23     qty = int(qty_box.get())
24     cart[item_id] = cart.get(item_id, 0) + qty
25     show_cart()
26

```

- Remove from Cart Function

This function allows the user to select an item inside the shopping cart box and delete it if they change their mind.

```

27 def remove_from_cart():
28     selected = cart_list.curselection()
29     if not selected:
30         messagebox.showwarning("No item", "Select an item in the")
31         return
32     item_id = list(cart.keys())[selected[0]]
33     del cart[item_id]
34     show_cart()
35

```

- Display and Refresh Cart Function

This function clears the cart box on the screen and recalculates the continuous subtotal of all items added .

```

36 def show_cart():
37     cart_list.delete(0, tk.END)
38     total = 0
39     for item_id, qty in cart.items():
40         name, price = items[item_id]
41         cost = price * qty
42         total += cost
43         cart_list.insert(tk.END, f"{name} x{qty} = ₹{cost:.2f}")
44     total_label.config(text=f"Subtotal: ₹{total:.2f}")
45

```

- Checkout and Bill Generation Function

This function verifies that the user entered a name and a correct 10-digit mobile number. It then calculates a **5% tax**, formats a text receipt, and displays it in a popup box.

```

46 def checkout():
47     name = name_entry.get().strip()
48     phone = phone_entry.get().strip()
49
50     if name == "":
51         messagebox.showerror("Error", "Please enter your name.")
52         return
53     if not (phone.isdigit() and len(phone) == 10):
54         messagebox.showerror("Error", "Phone number must be 10 d
55         return
56     if len(cart) == 0:
57         messagebox.showinfo("Empty cart", "Your cart is empty.")
58         return
59
60     subtotal = 0
61     bill = f"Customer: {name}\nPhone: {phone}\n\n"
62     for item_id, qty in cart.items():
63         item_name, price = items[item_id]
64         cost = price * qty
65         subtotal += cost
66         bill += f"{item_name} x{qty} = ₹{cost:.2f}\n"
67
68     tax = subtotal * 0.05
69     grand_total = subtotal + tax
70     bill += "-" * 28
71     bill += f"\nSubtotal: ₹{subtotal:.2f}"
72     bill += f"\nTax (5%): ₹{tax:.2f}"
73     bill += f"\nTotal Bill: ₹{grand_total:.2f}"
74
75     bill += f"\nTax (5%): ₹{tax:.2f}"
76     bill += f"\nTotal Bill: ₹{grand_total:.2f}"
77     bill += "\n\nThank you for shopping! Do visit again."
78
79     messagebox.showinfo("Final Bill", bill)
80     cart.clear()
81     show_cart()
82

```

- Graphical User Interface (GUI) Layout

This final section creates the main window, colors it pale green (**#eaf7ea**), sets up the labels, input text boxes, and action buttons.

```

80 window = tk.Tk()
81 window.title("Fresh Basket Grocery Store")
82 window.geometry("550x500")
83 window.config(bg="#eaf7ea")
84
85 tk.Label(window, text="FRESH BASKET GROCERY STORE",
86 | | | font=("Times New Roman", 16, "bold"), bg="#eaf7ea", f
87
88 details = tk.Frame(window, bg="#eaf7ea")
89 details.pack()
90 tk.Label(details, text="Name:", bg="#eaf7ea").grid(row=0, column=0,
91 name_entry = tk.Entry(details, width=25)
92 name_entry.grid(row=0, column=1, pady=3)
93 tk.Label(details, text="Phone:", bg="#eaf7ea").grid(row=1, column=0
94 phone_entry = tk.Entry(details, width=25)
95 phone_entry.grid(row=1, column=1, pady=3)
96
97 middle = tk.Frame(window, bg="#eaf7ea")
98 middle.pack(pady=10)
99
100 left = tk.Frame(middle, bg="#eaf7ea")
101 left.grid(row=0, column=0, padx=10)
102 tk.Label(left, text="Items", font=("Times New Roman", 12, "bold"), bg
103 item_list = tk.Listbox(left, width=22, height=10)
104 for name, price in items.values():
105 | item_list.insert(tk.END, f"{name} - ₹{price:.2f}")
106 item_list.pack()
107

```

```

108 qty_row = tk.Frame(left, bg="#eaf7ea")
109 qty_row.pack(pady=5)
110 tk.Label(qty_row, text="Qty:", bg="#eaf7ea").pack(side="left")
111 qty_box = tk.Spinbox(qty_row, from_=1, to=20, width=5, state="re
112 qty_box.pack(side="left")
113
114 tk.Button(left, text="Add to Cart", bg="#4caf50", fg="white",
115 | | | width=15, command=add_to_cart).pack()
116
117 right = tk.Frame(middle, bg="#eaf7ea")
118 right.grid(row=0, column=1, padx=10)
119 tk.Label(right, text="Your Cart", font=("Times New Roman", 12, "bol
120 cart_list = tk.Listbox(right, width=26, height=10)
121 cart_list.pack()
122 total_label = tk.Label(right, text="Subtotal: ₹0.00", bg="#eaf7ea
123 | | | | | font=("Times New Roman", 11, "bold"))
124 total_label.pack(pady=5)
125 tk.Button(right, text="Remove Item", bg="#e57373", fg="white",
126 | | | width=15, command=remove_from_cart).pack()
127
128 # Checkout
129 tk.Button(window, text="Checkout", bg="#2e7d32", fg="white",
130 | | | font=("Times New Roman", 12, "bold"), width=20, command=
131
132 window.mainloop()

```

8.Key Functions

Functions	Purpose
Add to cart	Grabs the index of the selected item

	from the menu listbox, matches it with the items dictionary key, reads the spinbox quantity, and increments the item count inside the cart dictionary .
Remove from cart	Clears out the selected item entirely from the active memory dictionary and calls the refresh UI routine.
checkout	Evaluates text fields using data validations (rejects blank names, requires 10 digits for phone, and confirms cart size > 0). It dynamically loops to print a cleanly formatted receipt string showing item breakdowns, flat tax computations, and totals inside a native message box .

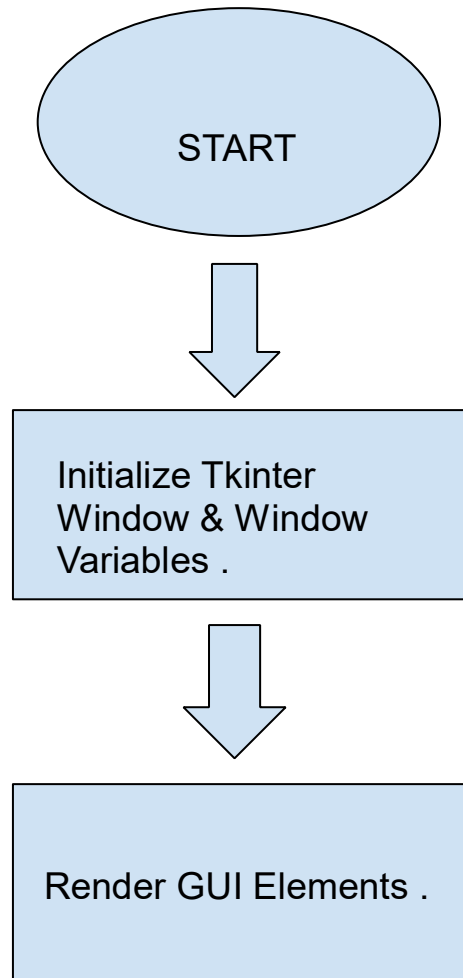
9. Functional Testing Approaches

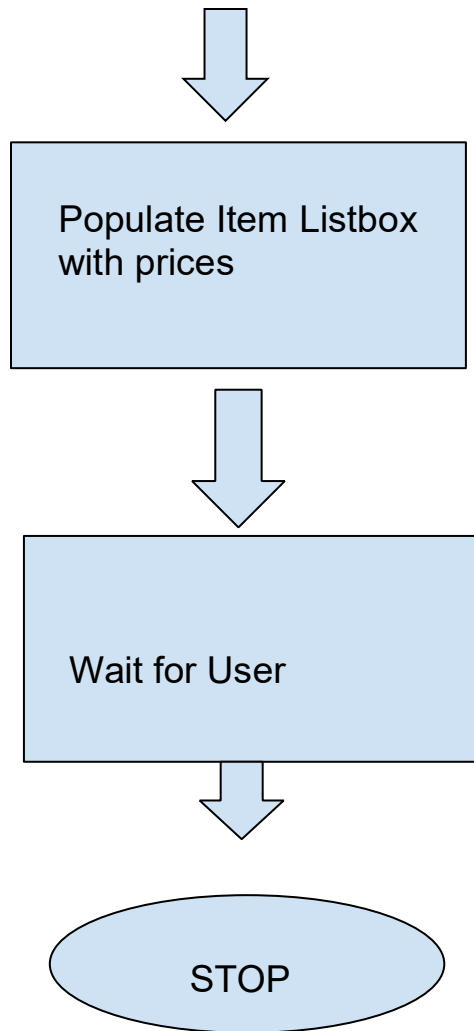
Functional testing verifies that the application features work exactly as defined in the source code requirements.

- Unit Testing: Focuses on validating individual Python functions in isolation. You can use Python's built-in `unittest` library to test core mathematical calculations. For example, pass a dummy cart dictionary to the system and verify that the 5% tax and grand total formulas yield 100% accurate results down to two decimal places.
- Boundary Value Analysis (BVA): Tests the limits of input fields. For the Phone Number box, test inputs of exactly 9 digits, 10 digits (valid), and 11

digits. For the Quantity Spinbox, verify how the system behaves at the minimum allowed quantity (1) and the maximum boundary (20)

10.Flowchart





11.Conclusion

The development of the **Fresh Basket Grocery Store Billing System** has been successfully completed using Python and the Tkinter GUI library. The project effectively demonstrates how standard retail operations can be digitized to save time and reduce manual calculation errors.

12.Key Takeaways from the Project:

- **Successful Automation:** The application successfully automates the entire billing workflow—from adding items and managing quantities to calculating a **5% tax rate** and generating the final bill.
- **Effective Data Validation:** By enforcing simple rules (like requiring a **10digit phone number** and preventing an empty checkout), the system ensures that user input errors are caught immediately.
- **Practical GUI Learning:** Developing this project helped in understanding how event-driven programming works, specifically how to link user actions (button clicks) to backend Python logic (updating dictionaries and text widgets).

In summary, the system provides a lightweight, clean, and reliable solution tailored for small business grocery management, fulfilling all its initial design objectives.

13.Project References & Bibliography

Vityarthi modules and lectures and python book